

Trilha de Node.js

Módulo 04: Desenvolvimento Web

Instruções para a melhor prática de Estudo

1. **Leia atentamente todo o conteúdo:** Antes de iniciar qualquer atividade, faça uma leitura detalhada do material fornecido na trilha, compreendendo os conceitos e os exemplos apresentados.
2. **Não se limite ao material da trilha:** Utilize o material da trilha como base, mas busque outros materiais de apoio, como livros, artigos acadêmicos, vídeos, e blogs especializados. Isso enriquecerá o entendimento sobre o tema.
3. **Explore a literatura:** Consulte livros e publicações reconhecidas na área, buscando expandir seu conhecimento além do que foi apresentado. A literatura acadêmica oferece uma base sólida para a compreensão de temas complexos.
4. **Realize todas as atividades propostas:** Conclua cada uma das atividades práticas e teóricas, garantindo que você esteja aplicando o conhecimento adquirido de maneira ativa.
5. **Evite o uso de Inteligência Artificial para resolução de atividades:** Utilize suas próprias habilidades e conhecimentos para resolver os exercícios. O aprendizado vem do esforço e da prática.
6. **Participe de debates:** Discuta os conteúdos estudados com professores, colegas e profissionais da área. O debate enriquece o entendimento e permite a troca de diferentes pontos de vista.
7. **Pratique regularmente:** Não deixe as atividades para a última hora. Pratique diariamente e revise o conteúdo com frequência para consolidar o aprendizado.
8. **Peça feedback:** Solicite o retorno dos professores sobre suas atividades e participe de discussões sobre os erros e acertos, utilizando o feedback para aprimorar suas habilidades.

Essas instruções são fundamentais para garantir um aprendizado profundo e eficaz ao longo das trilhas.

Módulo 4: Desenvolvimento Web com Node.js

Objetivo:

Capacitar os alunos a desenvolver aplicações backend robustas e organizadas com Node.js, utilizando o framework Express.js, integrando banco de dados MySQL e aplicando boas práticas de arquitetura, segurança e manutenção, além de compreender e utilizar o modelo de programação orientada a eventos por meio do EventEmitter.

4.1 - Introdução ao Express.js

O que é o Express.js?

- Um framework minimalista para Node.js que simplifica a criação de servidores e APIs.
- Oferece suporte para middleware, roteamento e integração com bancos de dados.

Instalação do Express.js

```
npm install express
```

Exemplo de Servidor Básico

```
const express = require('express');
require('dotenv').config();

const app = express();
const PORT = Number(process.env.PORT || 3000);

app.get('/', (req, res) => {
  res.send('Bem-vindo ao servidor Express.js!');
});

app.get('/status', (req, res) => {
  res.json({ ok: true, timestamp: new Date().toISOString() });
});

app.listen(PORT, () => {
  console.log(`Servidor rodando na porta ${PORT}`);
});
```

4.2 - Rotas HTTP

Tipos de Rotas

Método HTTP	Finalidade	Descrição prática
GET	Consulta	Usado para buscar dados do servidor
POST	Criação	Usado para enviar e criar dados no servidor
PUT	Atualização	Usado para atualizar dados existentes
DELETE	Exclusão	Usado para remover dados do servidor

Exemplo Prático

```
// GET /users
app.get('/users', (req, res) => {
  res.json({ message: 'Listando usuários' });
});

// POST /users
app.post('/users', (req, res) => {
  res.status(201).json({ message: 'Usuário criado', payload: req.body });
});

// PUT /users/:id
app.put('/users/:id', (req, res) => {
  res.json({ message: `Usuário ${req.params.id} atualizado`, payload: req.body });
});

// DELETE /users/:id
app.delete('/users/:id', (req, res) => {
  res.json({ message: `Usuário ${req.params.id} excluído` });
});
```

4.3 - Middlewares

Middleware é uma **função intermediária** que é executada **entre a requisição do cliente e a resposta do servidor**.

Na prática, ele é usado para **verificar, preparar ou bloquear** uma requisição **antes** que ela chegue à regra principal do sistema.

Para que isso é usado no dia a dia da empresa?

Middlewares são usados para:

- **Registrar logs** de requisições (quem acessou, quando e qual rota)
- **Validar dados** antes de processar uma ação
- **Verificar autenticação e permissões** (usuário logado, token válido)
- **Tratar erros de forma centralizada**

- **Padronizar respostas** da API

Ou seja, middleware evita repetir código e ajuda a manter a aplicação **organizada, segura e fácil de manter**.

Tipos de Middleware

- **Global:** Aplica-se a todas as rotas.
- **Específico:** Aplica-se a rotas selecionadas.

Exemplo Prático

```
// Middleware global
app.use((req, res, next) => {
  console.log(`Método: ${req.method}, URL: ${req.url}`);
  next();
});

// Middleware específico
app.use('/users', (req, res, next) => {
  console.log('Middleware aplicado apenas às rotas de /users');
  next();
});
```

4.4 - Integração com MySQL

```
require('dotenv').config();
const mysql = require('mysql2/promise');

/**
 * Pool de conexões (recomendado) – mais eficiente que createConnection para APIs.
 * - Reutiliza conexões
 * - Evita abrir/fechar conexão a cada requisição
 */
const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
  port: Number(process.env.DB_PORT || 3306),
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});

module.exports = pool;
```

Consulta Básica

```

require('dotenv').config();
const pool = require('./db');

(async () => {
  try {
    const [rows] = await pool.query('SELECT * FROM usuarios');
    console.log('✅ Resultado:', rows);
    process.exit(0);
  } catch (err) {
    console.error('❌ Falha na consulta:', err.message);
    process.exit(1);
  }
})();

```

4.5 - CRUD com MySQL

CRUD Completo

Operação	Ação no Sistema	O que faz na prática	Exemplo no dia a dia
Create	Criar	Insere novos dados no banco	Cadastrar um novo usuário, produto ou pedido
Read	Ler / Consultar	Busca dados já existentes	Listar usuários, consultar pedidos, visualizar relatórios
Update	Atualizar	Altera dados já cadastrados	Atualizar e-mail, alterar status de um pedido
Delete	Excluir	Remove dados do sistema	Excluir um usuário ou cancelar um registro

Exemplo de CRUD

Segue o link, nele conseguimos o acesso ao código fonte referente a um CRUD completo usando o MySQL: [CRUD - NodeJS](#)

4.6 - Segurança no Backend

Boas Práticas

- **Use prepared statements:** Evita SQL Injection.
- **Validação de entrada:** Certifique-se de que os dados recebidos sejam seguros.
- **Use variáveis de ambiente:** Não exponha credenciais no código.

Exemplo de Proteção

```
// Exemplo de consulta segura (prepared statement)
app.get('/usuarios/nome/:nome', async (req, res) => {
  const { nome } = req.params;

  try {
    const [rows] = await pool.execute('SELECT * FROM usuarios WHERE nome = ?', [nome]);
    return res.json(rows);
  } catch (err) {
    return res.status(500).json({ error: err.message });
  }
});

app.listen(PORT, () => console.log(`✅ Proteção SQL Injection rodando na porta ${PORT}`));
```

4.7 - Programação Orientada a Eventos

O `EventEmitter` é uma classe central no Node.js que fornece uma forma de lidar com eventos. Ele é usado para criar e gerenciar sistemas baseados em eventos, nos quais objetos podem emitir eventos e outras partes do código podem ouvir e reagir a esses eventos. Essa abordagem é fundamental em muitas aplicações Node.js, especialmente aquelas que requerem interação assíncrona.

A classe `EventEmitter` faz parte do módulo `events` do Node.js e segue o padrão de publicação/assinatura, onde um "emissor" emite um evento, e "ouvintes" (listeners) reagem a esse evento.

5 - Importando EventEmitter

Antes de usar o `EventEmitter`, você precisa importá-lo do módulo `events`:

```
const EventEmitter = require('events');
```

Principais Métodos do EventEmitter

1. `on(event, listener)`

Registra uma função que será executada **toda vez** que o evento acontecer.

Quando usar na empresa: Quando você precisa reagir sempre que algo ocorrer, como:

- registrar logs
- notificar usuários
- atualizar métricas

Exemplo: sempre que um usuário faz login, registrar no log.

2. `emit(event, [...args])`

Dispara um evento, executando **todos os listeners** associados a ele.

Quando usar na empresa: Quando algo importante acontece no sistema e outras partes precisam ser avisadas, sem criar dependência direta entre elas.

Exemplo: emitir um evento após um pagamento aprovado.

3. `once(event, listener)`

Registra um listener que será executado **apenas na primeira vez** que o evento ocorrer.

Quando usar na empresa: Quando a ação deve acontecer uma única vez, como:

- inicialização do sistema
- primeiro acesso do usuário
- configuração inicial

Exemplo: executar uma ação somente no primeiro login.

4. `removeListener(event, listener)` ou `off(event, listener)`

Remove um listener específico de um evento.

Quando usar na empresa: Quando uma funcionalidade não deve mais reagir a determinado evento, evitando:

- execuções desnecessárias
- vazamento de memória

Exemplo: parar de escutar eventos de um usuário desconectado.

5. `removeAllListeners([event])`

Remove **todos os listeners** de um evento específico ou de todos os eventos do sistema.

Quando usar na empresa: Quando um módulo é finalizado, reiniciado ou descarregado e você precisa garantir que não fique nenhum listener ativo.

Exemplo: limpar eventos ao encerrar uma sessão ou reiniciar um serviço.

6. listenerCount(event)

Retorna quantos listeners estão registrados para um determinado evento.

Quando usar na empresa: Para **debug e monitoramento**, ajudando a identificar:

- excesso de listeners
- possíveis vazamentos de memória

Exemplo: verificar se múltiplos módulos estão escutando o mesmo evento indevidamente.

Exemplo Básico de Uso

Aqui está um exemplo básico para entender como o `EventEmitter` funciona:

```
const myEmitter = new EventEmitter();

myEmitter.on('greet', (name) => {
  console.log(`Olá, ${name}!`);
});

myEmitter.emit('greet', 'João');
```

Exemplo Prático: Sistema de Login

Imagine um sistema de login onde queremos emitir eventos baseados no sucesso ou falha de autenticação:

```
class Auth extends EventEmitter {
  login(username, password) {
    // Lógica simulada de login
    if(username === 'admin' && password === '1234') {
      this.emit('loginSuccess', username);
    } else {
      this.emit('loginFailure', username);
    }
  }
}

const auth = new Auth();

auth.on('loginSuccess', (user) => console.log(`Usuário ${user} logado com sucesso!`));
auth.on('loginFailure', (user) => console.log(`Falha no login para o usuário: ${user}`));

auth.login('admin', '1234');
auth.login('user', 'wrongpass');
```

Exemplo Avançado: Chat em Tempo Real

No caso de um sistema de chat, você pode emitir eventos como `message` ou `userJoined`:

```
class ChatRoom extends EventEmitter {
  join(user) {
    this.emit('userJoined', user);
  }

  sendMessage(user, message) {
    this.emit('message', user, message);
  }
}

const chatRoom = new ChatRoom();

chatRoom.on('userJoined', (user) => {
  console.log(`${user} entrou na sala de chat.`);
});

chatRoom.on('message', (user, message) => {
  console.log(`${user} diz: ${message}`);
});

chatRoom.join('Alice');
chatRoom.sendMessage('Alice', 'Olá a todos!');
chatRoom.join('Bob');
chatRoom.sendMessage('Bob', 'Oi, Alice!');
```

Saída:

```
Alice entrou na sala de chat.
Alice diz: Olá a todos!
Bob entrou na sala de chat.
Bob diz: Oi, Alice!
```

Vantagens do EventEmitter

- **Facilidade na implementação de eventos:** Com poucos métodos, é possível criar sistemas robustos.
- **Flexibilidade:** Útil para cenários como sistemas de notificação, streams de dados ou interfaces interativas.
- **Reutilização:** Eventos podem ser facilmente gerenciados e reutilizados em diferentes partes de um aplicativo.

Boas Práticas

1. **Evitar vazamentos de memória:**
 - O Node.js exibe um aviso se mais de 10 listeners forem adicionados ao mesmo evento. Use `emitter.setMaxListeners(n)` para ajustar o limite.
2. **Remover listeners desnecessários:**
 - Listeners que não são mais úteis devem ser removidos para liberar recursos.
3. **Erro centralizado:**
 - Sempre adicione um listener para eventos de erro:

```
myEmitter.on('error', (err) => {
  console.error('Erro capturado:', err);
});
```

Atividades Teóricas

1. **Exploração da Documentação Oficial**
 - **Objetivo:** Familiarizar-se com a documentação do módulo `events` no Node.js.
 - **Atividade:** Pesquise a documentação oficial e responda:
 - i. Quais métodos principais a classe `EventEmitter` oferece?
 - ii. Qual é a diferença entre os métodos `on` e `once`?
 - iii. O que acontece se você tentar emitir um evento sem um listener?
2. **Diagrama de Fluxo de Eventos**
 - **Objetivo:** Entender o fluxo de um evento em um sistema baseado em eventos.
 - **Atividade:** Crie um diagrama de fluxo que mostre o processo de:
 - i. Registro de listeners.
 - ii. Emissão de eventos.
 - iii. Execução de callbacks.
3. **Questionário de Múltipla Escolha**
 - **Objetivo:** Testar o conhecimento básico sobre o `EventEmitter`.
 - **Atividade:** Responda perguntas como:
 - i. Qual método é usado para emitir eventos?
 - ii. O que faz o método `removeListener`?
 - iii. É possível registrar vários listeners para o mesmo evento?
4. **Análise de Código**
 - **Objetivo:** Identificar e explicar como o `EventEmitter` está sendo usado.
 - **Atividade:** Analise o seguinte código e explique cada parte:

```
const EventEmitter = require('events');
const emitter = new EventEmitter();
emitter.on('data', (info) => {
    console.log('Dado recebido:', info);
});
emitter.emit('data', { id: 1, message: 'Olá!' });
```

5. Estudo de Caso

- **Objetivo:** Relacionar o uso de eventos com casos reais.
- **Atividade:** Dê exemplos de aplicações práticas que poderiam usar o `EventEmitter`, como:
 - Streaming de áudio/vídeo.
 - Notificações em tempo real.
 - Controle de fluxos de dados em APIs.

Atividades Práticas

- **Criação de Eventos Simples**
 - **Objetivo:** Implementar e testar eventos básicos.
 - **Atividade:**
 - Crie um programa que emite um evento chamado `hello` e responde com "Hello, World!".
 - Modifique para receber o nome de uma pessoa e dizer "Hello, [nome]!".
- **Sistema de Registro de Log**
 - **Objetivo:** Implementar um sistema de logging usando o `EventEmitter`.
 - **Atividade:**
 - Crie um evento `log` que registra mensagens no console.
 - Adicione diferentes níveis de log: `info`, `warn` e `error`.
- **Simulação de Chat**
 - **Objetivo:** Simular um sistema de chat com eventos.
 - **Atividade:**
 - Crie uma classe `ChatRoom` que emite eventos quando usuários entram na sala e enviam mensagens.
 - Teste emitindo eventos como `userJoined` e `message`.
- **Eventos com Múltiplos Listeners**
 - **Objetivo:** Demonstrar como vários listeners podem ser registrados para o mesmo evento.
 - **Atividade:**
 - Crie um evento `dataReceived` com dois listeners:
 - Um listener que salva os dados em um arquivo.
 - Outro que exibe os dados no console.

- **Gerenciamento de Erros com Eventos**
 - **Objetivo:** Praticar o uso do evento `error`.
 - **Atividade:**
 - Implemente um `EventEmitter` que emite um evento `error` quando ocorre um problema.
 - Crie um listener para tratar esses erros e exibir mensagens apropriadas no console
-

Lista de Exercícios

Questões Teóricas

1. O que é o Express.js e quais suas principais vantagens?
2. Quais são os quatro principais tipos de rotas HTTP?
3. Explique o conceito de middleware no Express.js.
4. Como criar uma conexão básica entre Node.js e MySQL?
5. O que é um prepared statement e como ele protege contra SQL Injection?
6. Qual é a estrutura básica de um servidor Express.js?
7. Explique a diferença entre rotas globais e rotas específicas.
8. Liste três boas práticas ao manipular dados em bancos de dados.
9. Por que é importante usar variáveis de ambiente em aplicações Node.js?
10. Como tratar erros em consultas SQL dentro do Node.js?

Questões Práticas

- Crie um servidor Express.js que responda à rota `/` com "Hello, World!".
- Implemente uma rota GET que liste todos os usuários de um banco MySQL.
- Adicione um middleware que registre o método HTTP e a URL de cada requisição.
- Crie uma rota POST que insira um novo usuário no banco de dados.
- Implemente uma rota PUT para atualizar o nome de um usuário pelo ID.
- Crie uma rota DELETE que exclua um usuário pelo ID.
- Use prepared statements para consultar um usuário pelo nome de forma segura.
- Configure variáveis de ambiente para conexão com o banco.
- Adicione tratamento de erros para uma consulta SQL que pode falhar.
- Estruture um CRUD completo com separação de módulos em arquivos diferentes.